

GraphQL + Apollo + Federation – Proyecto "GraphiQloud".

Introducción.

¿Por qué GraphQL y no REST?

En la actualidad, las aplicaciones modernas consumen datos desde múltiples servicios (usuarios, productos, pedidos, pagos, etc.). Con REST, tendríamos que hacer múltiples requests o tener un endpoint que devuelve datos que no siempre necesitamos.

GraphQL permite:

- Pedir exactamente los datos que necesitamos.
- Obtener todos los datos en una sola consulta.
- Evolucionar la API sin versionado.

Federation extiende GraphQL para arquitectura de microservicios: cada servicio aporta sus propios tipos y resolvers, y un Gateway los une en un único endpoint.

Analogía: REST es como pedir en 3 ventanillas distintas (usuarios, productos, pedidos). GraphQL es como una ventanilla única donde podés pedir un combo personalizado. Federation es como tener varias sucursales que colaboran.

GraphQL – Conceptos base.

- **Schema:** define tipos y operaciones (Query, Mutation, Subscription).
- **Type:** objeto con campos tipados.
- **Resolver:** función que devuelve datos para un campo.
- **Query:** lectura de datos.
- **Mutation:** escritura/modificación.
- **Subscription:** eventos en tiempo real (WebSockets).

Apollo Server + Federation.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- Apollo Server: librería para montar servidor GraphQL en Node.js.
- @apollo/subgraph**: permite crear subgraphs (servicios federados).
- @apollo/gateway**: orquesta los subgraphs en un endpoint único.

DataLoader – Evitar problema N+1.

Supongamos que pedimos 100 usuarios y cada uno tiene una lista de productos. Sin DataLoader, haríamos 1 consulta para usuarios + 100 consultas de productos. DataLoader agrupa las consultas.

Apollo Client – Caché avanzada.

- Normaliza los datos por `\_\_typename` e `id`.
- Permite actualizar la caché después de mutations sin re-fetch.

Subscriptions en tiempo real.

- Usan WebSockets.
- Útiles para: notificaciones, chat, stock actualizado, pedidos en vivo.

Ejemplo Práctico.

Tema: "Gestión de votos en una plataforma de películas".

Escenario:

- Servicio `movies`: título, año.
- Servicio `reviews`: comentarios, puntuación.
- Servicio `voting`: votos en tiempo real.

Código demostrativo (estructura rápida):

Subgraph – movies

```
extend type Query {  
  movie(id: ID!): Movie  
}
```

```
type Movie @key(fields: "id") {  
  id: ID!  
  title: String!  
  year: Int!  
}
```

Subgraph – voting (con subscription)

```
type Vote {  
  movieId: ID!  
  userId: ID!  
  value: Int!  
}
```

```
type Subscription {  
  voteAdded(movieId: ID!): Vote  
}
```

Gateway – une ambos.

DataLoader en `reviews` para evitar N+1:

```
const movieLoader = new DataLoader(async (ids) => {  
  const movies = await db.movies.find({ id: { $in: ids } });  
  return ids.map(id => movies.find(m => m.id === id));  
});
```

Proyecto principal: "GraphiQloud – Conectando servicios en la estratosfera digital".

- `users-auth`: autenticación y perfil de usuario.
- `products-catalog`: productos con stock, precio, categoría.
- `orders-tracking`: pedidos, estado de envío, tiempo estimado.
- `notifications-realtime`: alertas cuando un pedido cambia de estado (subscriptions).

La plataforma simula cómo una empresa grande (Netflix, Uber, Shopify) federaría sus APIs internas.

Objetivos.

1. Diseñar schemas GraphQL modulares usando `@key`` y `@external``.
2. Implementar Apollo Server con Federation (gateway + subgraphs).
3. Configurar Apollo Client en frontend con caché avanzada (`InMemoryCache``, `cache.modify``).
4. Usar DataLoader para resolver el problema N+1 en al menos un resolver.
5. Implementar Subscriptions en tiempo real (WebSockets) para notificaciones.

Requisitos técnicos mínimos.

- Node.js + Express o Apollo Server standalone.
- Subgraphs: mínimo 3 servicios federados.
- Gateway con Apollo Gateway.
- Frontend (React, Vue o HTML/JS vanilla con Apollo Client).
- Al menos una subscription funcionando.
- Uso demostrado de DataLoader.
- Repositorio GitHub con README claro.
- Despliegue en Vercel (o Railway/Render) del gateway + subgraphs (o al menos del frontend y un subgraph).

Criterios de evaluación.

Criterio	%	Condición
Entrega parcial en clase (60% obligatorio)	40%	Al finalizar la clase, deben subir ZIP o link a Drive con el código avanzado hasta ese momento + enlace GitHub
Funcionamiento del Gateway y 3 subgraphs	10%	Endpoint único responde consultas.
Resolución N+1 con DataLoader	10%	Explicación en código y muestra de mejora.
Subscription en tiempo real	10%	Ejemplo claro (ej. notificación al crear pedido).
Frontend con caché avanzada	5%	Uso de <code>cache.readQuery`</code> / <code>cache.modify`</code> .
Documentación técnica (informe final)	10%	Ver sección informe
Despliegue funcional (Vercel u otro)	5%	Link accesible.
Buenas prácticas (código limpio, commits, Live Share)	10%	Participación activa durante la clase.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- El 60% debe estar avanzado o completado durante las 4 horas de clase. El 40% restante puede finalizarse como tarea hasta el martes anterior a la siguiente clase.

Informe técnico (documentación final – obligatorio).

Cada grupo debe entregar un documento (PDF o Markdown en el repo) que contenga:

1. Portada con integrantes, fecha, nombre del proyecto.
2. Diagrama de arquitectura (subgraphs, gateway, cliente).
3. Explicación de decisiones técnicas (por qué Federation, cómo usaron DataLoader).
4. Capturas de consultas GraphQL exitosas y subscription funcionando.
5. Instrucciones para levantar el proyecto localmente.
6. Enlace al despliegue en Vercel.
7. Reflexión: ¿Qué fue lo más desafiante? ¿Cómo resolvieron el problema N+1?

Entregables finales (antes del martes siguiente).

- Repositorio GitHub público (cada grupo). Informe técnico dentro del repo.
- Aplicación desplegada (Vercel).
- Enlace de Live Share compartido por comentario privado en el aula virtual ANTES de empezar a codificar en clase (para observación docente).
- Al finalizar la clase: subir ZIP o link a Drive con el estado del proyecto + enlace a GitHub (parcial).

Ejemplo de estructura de monorepo:

```
graphiqcloud/  
├── gateway/  
├── subgraphs/  
│   ├── users/  
│   ├── products/  
│   ├── orders/  
│   └── notifications/  
├── frontend/  
├── README.md  
└── docker-compose.yml (opcional)
```